

LAPORAN PRAKTIKUM

STRUKTUR DATA

“ALGORITMA SORTING”



disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillah rabbil ‘alamin, segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga saya dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “*ALGORITMA SORTING*”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam wawasan saya mengenai penerapan konsep struktur data, khususnya *Algoritma Sorting* dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan data yang bersifat dinamis.

Saya menyampaikan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Algoritma dan Pemrograman, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga saya sampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan dan keterbatasan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum di masa mendatang. Meskipun demikian, saya berharap laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	2
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	3
2.1 Program <i>IntertionSort</i>	3
2.2 Program <i>SelectionSort</i>	8
2.3 Program <i>BubbleSort</i>	13
2.4 Program <i>InsertionSortGUI</i>	18
BAB III KESIMPULAN.....	32
3.1 Kesimpulan	32
DAFTAR PUSTAKA	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran struktur data, mahasiswa tidak hanya mempelajari konsep dasar pemrograman, tetapi juga mempelajari algoritma untuk mengelola dan mengurutkan data secara efisien. Salah satu materi penting dalam struktur data adalah algoritma *sorting* atau pengurutan data.

Sorting merupakan proses mengurutkan data berdasarkan urutan tertentu, seperti dari nilai terkecil ke terbesar atau secara alfabetis. Algoritma sorting banyak digunakan dalam berbagai aplikasi komputer karena dapat mempermudah pencarian, pengelompokan, dan pengolahan data. Pada praktikum ini digunakan tiga algoritma *sorting*, yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.

Insertion Sort bekerja dengan cara menyisipkan elemen pada posisi yang tepat di bagian *array* yang sudah terurut. *Selection Sort* bekerja dengan mencari nilai terkecil kemudian menukarnya ke posisi yang sesuai. Sedangkan *Bubble Sort* bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya salah.

Pada praktikum ini, algoritma *sorting* diimplementasikan menggunakan bahasa Java. Selain program berbasis *console*, praktikum juga dikembangkan menggunakan Java *GUI* (*Swing*) untuk menampilkan proses pengurutan data secara visual dan interaktif. Program *GUI* memungkinkan pengguna memasukkan data *array* secara manual, kemudian melihat proses pengurutan langkah demi langkah menggunakan algoritma *Insertion Sort*.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar algoritma *sorting*, cara kerja masing-masing algoritma, serta implementasi visualisasi *sorting* menggunakan Java *GUI* (*Swing*).

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma *sorting* dalam struktur data.
2. Mengetahui cara kerja algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
3. Mempelajari implementasi algoritma *sorting* menggunakan bahasa Java.
4. Mengimplementasikan proses pengurutan data menggunakan *array*.
5. Membuat program *GUI* menggunakan Java *Swing* untuk visualisasi proses *sorting*.
6. Menampilkan langkah-langkah proses *sorting* secara bertahap pada program *GUI*.
7. Melatih kemampuan dalam membuat program struktur data dan algoritma menggunakan Java.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman tentang algoritma *sorting* dalam struktur data.
2. Membantu memahami proses pengurutan data menggunakan *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
3. Melatih kemampuan membuat program Java berbasis *console* dan *GUI (Swing)*.
4. Meningkatkan kemampuan logika dalam proses pengurutan data.
5. Membantu memahami visualisasi langkah-langkah proses *sorting*.
6. Membantu memahami implementasi *array* dalam proses pengolahan data.
7. Melatih kemampuan membuat program interaktif menggunakan Java *GUI*.

BAB II

PEMBAHASAN

2.1 Program *IntertionSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan7_2511532025;

public class IntertionSort_2511532025 {
    public static void InsertionSort_2511532025 (int[] arr) {
        int n_2025 = arr.length;
        for (int i_2025 = 1; i_2025 < n_2025; i_2025++) {
            int key_2025 = arr[i_2025];
            int j_2025 = i_2025 - 1;
            while (j_2025 >= 0 && arr[j_2025] > key_2025)
            {
                arr[j_2025 + 1] = arr[j_2025];
                j_2025--;
            }
            arr[j_2025 + 1] = key_2025;
        }
    }

    public static void main (String[] args) {
        int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2025 = arr.length;
        System.out.printf ("array yang belum terurut:
\n");
        for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
            System.out.print(arr[i_2025] + " ");
        System.out.println("");
        InsertionSort_2511532025 (arr);
        System.out.printf ("array yang terurut: \n");
        for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
            System.out.print(arr[i_2025] + " ");
        System.out.println("");
    }
}
```

Kode Program 2.1 Program *IntertionSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Insertion Sort*. Algoritma ini bekerja dengan cara mengambil satu elemen dari *array* kemudian membandingkannya dengan elemen sebelumnya hingga posisi yang sesuai ditemukan.

Setelah itu elemen akan disisipkan pada posisi yang benar sehingga bagian *array* di sebelah kiri selalu dalam kondisi terurut.

Method InsertionSort_2511532025() digunakan untuk melakukan proses pengurutan data. Program melakukan perulangan dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut. Nilai pada indeks saat ini disimpan ke dalam *variabel key_2025*, kemudian dibandingkan dengan elemen sebelumnya. Jika elemen sebelumnya lebih besar, maka elemen tersebut akan digeser ke kanan hingga ditemukan posisi yang tepat untuk *key_2025*.

Pada *method main()*, program membuat *array* berisi data 23, 78, 45, 8, 32, 56, 1. Selanjutnya program menampilkan isi *array* sebelum diurutkan, kemudian memanggil *method InsertionSort_2511532025()* untuk melakukan pengurutan data, dan akhirnya menampilkan *array* setelah terurut.

2.1.1 Langkah Kerja Program *IntertionSort*

1. Membuat *class IntertionSort_2511532025*

```
public class IntertionSort_2511532025 {
```

Kode Program 2.2 *Class IntertionSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Insertion Sort*.

2. Membuat *method InsertionSort_2511532025*

```
public static void InsertionSort_2511532025 (int[]  
arr) {
```

Kode Program 2.3 *Method InsertionSort*

Method ini digunakan untuk melakukan proses pengurutan *array* menggunakan algoritma *Insertion Sort*.

3. Mengambil panjang *array*

```
int n_2025 = arr.length;
```

Kode Program 2.4 Panjang *Array*

Syntax ini digunakan untuk mengambil jumlah elemen pada *array*.

4. Melakukan perulangan *array*

```
for (int i_2025 = 1; i_2025 < n_2025;  
i_2025++) {
```

Kode Program 2.5 Perulangan *Array*

Perulangan dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut.

5. Menyimpan nilai *key* dan *indeks* sebelumnya

```
int key_2025 = arr[i_2025];  
int j_2025 = i_2025 - 1;
```

Kode Program 2.6 Menentukan *Key*

Syntax ini digunakan untuk menyimpan nilai yang akan dibandingkan dan menentukan posisi elemen sebelumnya.

6. Membandingkan dan menggeser elemen

```
while (j_2025 >= 0 && arr[j_2025] >  
key_2025) {  
    arr[j_2025 + 1] = arr[j_2025];  
    j_2025--;  
}
```

Kode Program 2.7 Prose Perbandingan

Syntax ini digunakan untuk membandingkan elemen sebelumnya dengan nilai *key_2025*. Jika elemen sebelumnya lebih besar, maka elemen akan digeser ke kanan.

7. Menempatkan *key* pada posisi yang tepat

```
        arr[j_2025 + 1] = key_2025;
    }
}
```

Kode Program 2.8 Menempatkan *Key*

Syntax ini digunakan untuk menempatkan nilai *key_2025* pada posisi yang sesuai setelah proses pergeseran selesai.

8. Membuat *method main*

```
public static void main (String[] args) {
```

Kode Program 2.9 *Method Main*

Method main() merupakan method utama yang digunakan untuk menjalankan program.

9. Membuat *array data*

```
int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
int n_2025 = arr. length;
```

Kode Program 2.10 Membuat *Array Data*

Array ini digunakan sebagai data yang akan diurutkan menggunakan algoritma *Insertion Sort*.

10. Menampilkan *array* sebelum diurutkan

```
        System.out.printf ("array yang belum
terurut: \n");
        for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
            System.out.print(arr[i_2025] + "
");
        System.out.println("");
```

Kode Program 2.11 Menampilkan *Array* Awal

Syntax ini digunakan untuk menampilkan isi *array* sebelum dilakukan proses pengurutan.

11. Memanggil *method InsertionSort_2511532025*

```
InsertionSort_2511532025 (arr);
```

Kode Program 2.12 Memanggil *Method Sorting*

Syntax ini digunakan untuk menjalankan proses pengurutan data.

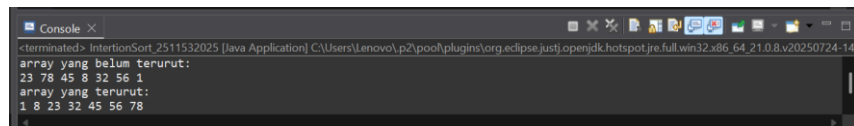
12. Menampilkan *array* setelah diurutkan

```
System.out.printf ("array yang terurut:\n");  
for (int i_2025 = 0; i_2025 < n_2025;  
i_2025++)  
    System.out.print(arr[i_2025] + "  
");  
    System.out.println("");  
}  
}
```

Kode Program 2.13 Menampilkan Hasil *Sorting*

Syntax ini digunakan untuk menampilkan hasil *array* setelah proses pengurutan selesai.

2.1.2 Hasil Program *IntertionSort*



Gambar 2.1 Hasil Program *InsertionSort*

Berdasarkan hasil eksekusi program, algoritma *Insertion Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi *array* adalah:

23 78 45 8 32 56 1

Setelah *method InsertionSort_2511532025()* dijalankan, *array* berhasil diurutkan menjadi:

1 8 23 32 45 56 78

Hasil tersebut menunjukkan bahwa algoritma *Insertion Sort* bekerja dengan cara membandingkan setiap elemen dengan elemen sebelumnya, kemudian menggeser elemen yang lebih besar hingga ditemukan posisi yang tepat. Proses ini dilakukan berulang sampai seluruh elemen *array* tersusun secara terurut dari nilai terkecil ke terbesar.

2.2 Program *SelectionSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan7_2511532025;

public class SelectionSort_2511532025 {
    public static void SelectionSort_2511532025 (int[] arr) {
        int n_2025 = arr.length;
        for (int i_2025 = 0; i_2025 < n_2025; i_2025++) {
            int minIndex = i_2025;
            for (int j_2025 = i_2025 + 1; j_2025 <
n_2025; j_2025++) {
                if (arr[j_2025] < arr[minIndex]) {
                    minIndex = j_2025;
                }
            }
            int temp_2025 = arr[i_2025];
            arr[i_2025] = arr[minIndex];
            arr[minIndex] = temp_2025;
        }
    }
    public static void main (String[] args) {
        int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2025 = arr. length;
        System.out.printf ("array yang belum terurut:
\n");
        for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
            System.out.print(arr[i_2025] + " ");
        System.out.println(""); ;
        SelectionSort_2511532025(arr);
        System.out.printf ("array yang terurut: \n");
```

```

        for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
            System.out.print(arr[i_2025] + " ");
            System.out.println("");
        }
    }
}

```

Kode Program 2.14 Program *SelectionSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Selection Sort*. Algoritma ini bekerja dengan cara mencari nilai terkecil pada *array*, kemudian menukarnya dengan elemen pada posisi awal. Proses ini dilakukan berulang hingga seluruh data tersusun secara terurut.

Method SelectionSort_2511532025() digunakan untuk melakukan proses pengurutan *array*. Pada setiap perulangan, program menentukan indeks dengan nilai terkecil menggunakan variabel *minIndex*. Setelah nilai terkecil ditemukan, program akan menukar posisi elemen tersebut dengan elemen pada indeks saat ini menggunakan variabel sementara *temp_2025*.

Pada *method main()*, program membuat *array* berisi data 23, 78, 45, 8, 32, 56, 1. Selanjutnya program menampilkan isi *array* sebelum diurutkan, kemudian memanggil *method SelectionSort_2511532025()* untuk melakukan proses pengurutan, dan akhirnya menampilkan *array* setelah data berhasil diurutkan.

2.2.1 Langkah Kerja Program *SelectionSort*

1. Membuat *class SelectionSort_2511532025*

```

public class SelectionSort_2511532025 {

```

Kode Program 2.15 *Class SelectionSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Selection Sort*.

2. Membuat *method SelectionSort_2511532025*

```
public static void SelectionSort_2511532025(int[]  
arr) {
```

Kode Program 2.16 *Method SelectionSort*

Method ini digunakan untuk melakukan proses pengurutan *array* menggunakan algoritma *Selection Sort*.

3. Mengambil panjang *array*

```
int n_2025 = arr.length;
```

Kode Program 2.17 Panjang *Array*

Syntax ini digunakan untuk mengambil jumlah elemen pada *array*.

4. Melakukan perulangan *array*

```
for (int i_2025 = 0; i_2025 < n_2025;  
i_2025++) {
```

Kode Program 2.18 Perulangan *Array*

Perulangan ini digunakan untuk menentukan posisi elemen yang akan dibandingkan dan ditukar.

5. Menentukan indeks nilai terkecil

```
int minIndex = i_2025;
```

Kode Program 2.19 Menentukan *Minimum Index*

Syntax ini digunakan untuk menyimpan indeks elemen terkecil sementara pada setiap proses perulangan.

6. Membandingkan elemen *array*

```
for (int j_2025 = i_2025 + 1; j_2025 <  
n_2025; j_2025++) {  
    if (arr[j_2025] < arr[minIndex])  
    {
```

```

        minIndex = j_2025;
    }
}

```

Kode Program 2.20 Kode Program Proses Perbandingan

Syntax ini digunakan untuk mencari nilai terkecil pada bagian array yang belum terurut.

7. Menukar data *array*

```

        int temp_2025 = arr[i_2025];
        arr[i_2025] = arr[minIndex];
        arr[minIndex] = temp_2025;
    }
}

```

Kode Program 2.21 Menukar Data

Syntax ini digunakan untuk menukar elemen saat ini dengan elemen terkecil yang telah ditemukan.

8. Membuat *method main*

```

public static void main (String[] args) {

```

Kode Program 2.22 *Method Main*

Method main() merupakan method utama yang digunakan untuk menjalankan program.

9. Membuat *array data*

```

        int arr[] = { 23, 78, 45, 8, 32, 56, 1
    };
        int n_2025 = arr.length;

```

Kode Program 2.23 Membuat *Array Data*

Array ini digunakan sebagai data yang akan diurutkan menggunakan algoritma *Selection Sort*.

10. Menampilkan *array* sebelum diurutkan

```
System.out.printf ("array yang belum
terurut: \n");
for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
    System.out.print(arr[i_2025] + "
");
System.out.println(""); ;
```

Kode Program 2.24 Menampilkan *Array* Awal

Syntax ini digunakan untuk menampilkan isi *array* sebelum dilakukan proses pengurutan.

11. Memanggil *method SelectionSort_2511532025*

```
SelectionSort_2511532025(arr);
```

Kode Program 2.25 Memanggil *Method Sorting*

Syntax ini digunakan untuk menjalankan proses pengurutan data.

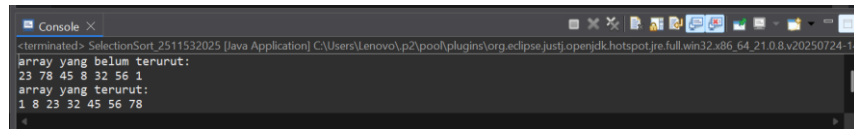
12. Menampilkan *array* setelah diurutkan

```
System.out.printf ("array yang terurut:
\n");
for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
    System.out.print(arr[i_2025] + "
");
System.out.println("");
}
```

Kode Program 2.26 Menampilkan Hasil *Sorting*

Syntax ini digunakan untuk menampilkan hasil *array* setelah proses pengurutan selesai.

2.2.2 Hasil Program *SelectionSort*



```
<terminated> SelectionSort_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-14
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Gambar 2.2 Hasil Program *SelectionSort*

Berdasarkan hasil eksekusi program, algoritma *Selection Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi *array* adalah:

23 78 45 8 32 56 1

Setelah *method SelectionSort_2511532025()* dijalankan, *array* berhasil diurutkan menjadi:

1 8 23 32 45 56 78

Hasil tersebut menunjukkan bahwa algoritma *Selection Sort* bekerja dengan cara mencari elemen terkecil dari bagian *array* yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal. Proses ini dilakukan secara berulang hingga seluruh elemen *array* tersusun secara terurut dari nilai terkecil ke terbesar.

2.3 Program *BubbleSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan7_2511532025;

public class BubbleSort_2511532025 {
    public static void bubbleSort_2025(int[] arr_2025) {
        int n_2025 = arr_2025.length;
        for(int i_2025 = 0; i_2025 < n_2025; i_2025++) {
            for (int j_2025 = 0; j_2025 < n_2025 - i_2025
- 1; j_2025++) {
                if (arr_2025[j_2025] > arr_2025[j_2025
+ 1]) {
                    int temp_2025 =
arr_2025[j_2025];
                    arr_2025[j_2025] =
arr_2025[j_2025 + 1];
                    arr_2025[j_2025 + 1] =
temp_2025;
                    //
System.out.println("data: " + arr[j_2025] + " " + arr[j_2025+1]);
                }
            }
        }
    }
}
```



```

    }
}

}

public static void main(String[] args) {
    int arr_2025[] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_2025 = arr_2025.length;
    System.out.print ("array yang belum terurut:");
    for (int i_2025 = 0; i_2025 < n_2025; i_2025++)
        System.out.print (arr_2025[i_2025] + " ");
    System.out.println("");
    bubbleSort_2025(arr_2025);
    System.out.print ("array yang terurut menggunakan
BubbleSort:");
    for (int i_2025 = 0; i_2025 < n_2025; i_2025++)
        System.out.print (arr_2025[i_2025] + " ");
    System.out.println("");
}
}

```

Kode Program 2.27 Program *BubbleSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Bubble Sort*. Algoritma ini bekerja dengan cara membandingkan dua elemen yang berdekatan, kemudian menukarnya apabila posisi elemen tidak sesuai urutan. Proses ini dilakukan secara berulang hingga seluruh elemen *array* tersusun secara terurut.

Method bubbleSort_2025() digunakan untuk melakukan proses pengurutan data. Program melakukan perulangan untuk membandingkan setiap elemen *array* dengan elemen di sebelah kanannya. Jika elemen sebelah kiri lebih besar dibandingkan elemen sebelah kanan, maka kedua elemen akan ditukar posisinya menggunakan variabel sementara.

Pada *method main()*, program membuat *array* berisi data 23, 78, 45, 8, 32, 56, 1. Selanjutnya program menampilkan isi *array* sebelum diurutkan, kemudian memanggil *method bubbleSort_2025()* untuk melakukan proses pengurutan data, dan akhirnya menampilkan *array* setelah terurut.

2.3.1 Langkah Kerja Program *BubbleSort*

1. Membuat *class BubbleSort_2511532025*

```
public class BubbleSort_2511532025 {
```

Kode Program 2.28 *Class BubbleSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Bubble Sort*.

2. Membuat *method bubbleSort_2025*

```
public static void bubbleSort_2025(int[] arr_2025) {
```

Kode Program 2.29 *Method BubbleSort*

Method ini digunakan untuk melakukan proses pengurutan *array* menggunakan algoritma *Bubble Sort*.

3. Mengambil panjang *array*

```
int n_2025 = arr_2025.length;
```

Kode Program 2.30 Panjang *Array*

Syntax ini digunakan untuk mengambil jumlah elemen pada *array*.

4. Melakukan perulangan *array*

```
for(int i_2025 = 0; i_2025 < n_2025;  
i_2025++) {
```

Kode Program 2.31 Perulangan Pertama

Perulangan ini digunakan untuk menentukan jumlah proses pengurutan yang dilakukan pada *array*.

5. Membandingkan elemen *array*

```
for (int j_2025 = 0; j_2025 < n_2025 - i_2025 - 1; j_2025++) {
```

Kode Program 2.32 Perulangan Kedua

Perulangan ini digunakan untuk membandingkan elemen yang berdekatan pada *array*.

6. Mengecek dan menukar data

```
if (arr_2025[j_2025] > arr_2025[j_2025 + 1]) {
    int temp_2025 = arr_2025[j_2025];
    arr_2025[j_2025] = arr_2025[j_2025 + 1];
    arr_2025[j_2025 + 1] = temp_2025;
    //
    System.out.println("data: " + arr[j_2025] + " " + arr[j_2025 + 1]);
}
}
```

Kode Program 2.33 Proses Pertukaran Data

Syntax ini digunakan untuk membandingkan dua elemen yang berdekatan. Jika elemen kiri lebih besar dari elemen kanan, maka kedua elemen akan ditukar posisinya.

7. Membuat *method main*

```
public static void main(String[] args) {
    int arr_2025[] = { 23, 78, 45, 8, 32, 56, 1 };
};
```

Kode Program 2.34 *Method Main*

Method main() merupakan *method* utama yang digunakan untuk menjalankan program.

8. Membuat *array data*

```
int n_2025 = arr_2025.length;
```

Kode Program 2.35 Membuat *Array Data*

Array ini digunakan sebagai data yang akan diurutkan menggunakan algoritma *Bubble Sort*.

9. Menampilkan *array* sebelum diurutkan

```
System.out.print ("array yang belum
terurut:");
for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
    System.out.print (arr_2025[i_2025] + "
");
System.out.println("");
```

Kode Program 2.36 Menampilkan *Array Awal*

Syntax ini digunakan untuk menampilkan isi *array* sebelum dilakukan proses pengurutan.

10. Memanggil *method bubbleSort_2025*

```
bubbleSort_2025(arr_2025);
```

Kode Program 2.37 Memanggil *Method Sorting*

Syntax ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Bubble Sort*.

11. Menampilkan *array* setelah diurutkan

```
System.out.print ("array yang terurut
menggunakan BubleSort:");
for (int i_2025 = 0; i_2025 < n_2025;
i_2025++)
    System.out.print (arr_2025[i_2025] + "
");
```

```

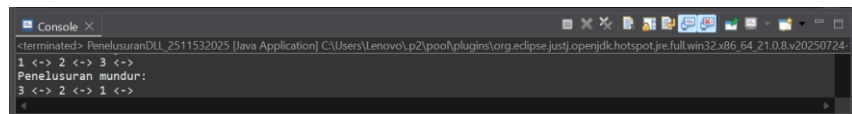
        System.out.println("");
    }
}

```

Kode Program 2.38 Menampilkan Hasil *Sorting*

Syntax ini digunakan untuk menampilkan hasil *array* setelah proses pengurutan selesai.

2.3.2 Hasil Program *BubbleSort*



Gambar 2.3 Hasil Program *BubbleSort*

Berdasarkan hasil eksekusi program, algoritma *Bubble Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi *array* adalah:

23 78 45 8 32 56 1

Setelah *method bubbleSort_2025()* dijalankan, *array* berhasil diurutkan menjadi:

1 8 23 32 45 56 78

Hasil tersebut menunjukkan bahwa algoritma *Bubble Sort* bekerja dengan cara membandingkan dua elemen yang berdekatan kemudian menukarnya apabila urutannya salah. Proses ini dilakukan secara berulang hingga seluruh elemen *array* tersusun secara terurut dari nilai terkecil ke terbesar.

2.4 Program *InsertionSortGUI*

Kode Program yang dipraktikkan adalah sebagai berikut:

```

package pekan7_2511532025;

import java.awt.BorderLayout;
import java.awt.Color;

```

```

import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511532025 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2025;
    private JLabel[] labelArray_2025;
    private JButton stepButton_2025, resetButton_2025,
setButton_2025;
    private JTextField inputField_2025;
    private JPanel panelArray_2025;
    private JTextArea stepArea_2025;

    private int i_2025 = 1, j_2025;
    private boolean sorting_2025 = false;
    private int stepCount_2025 = 1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532025() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
        inputField_2025 = new JTextField(30);
        setButton_2025 = new JButton("Set Array");
        inputPanel_2025.add(new JLabel("Masukkan angka
(pisahkan dengan koma):"));
        inputPanel_2025.add(inputField_2025);
        inputPanel_2025.add(setButton_2025);

        // Panel array visual
        panelArray_2025 = new JPanel();
        panelArray_2025.setLayout(new FlowLayout());

        // Panel kontrol

```

```

        JPanel controlPanel_2025 = new JPanel();
        stepButton_2025 = new JButton("Langkah
Selanjutnya");
        resetButton_2025 = new JButton("Reset");
        stepButton_2025.setEnabled(false);
        controlPanel_2025.add(stepButton_2025);
        controlPanel_2025.add(resetButton_2025);

        // Area teks untuk log langkah-langkah
        stepArea_2025 = new JTextArea(8, 60);
        stepArea_2025.setEditable(false);
        stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
        JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025);

        // Tambahkan panel ke frame
        add(inputPanel_2025, BorderLayout.NORTH);
        add(panelArray_2025, BorderLayout.CENTER);
        add(controlPanel_2025, BorderLayout.SOUTH);
        add(scrollPane_2025, BorderLayout.EAST);

        // Event Set Array
        setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

        // Event Langkah Selanjutnya
        stepButton_2025.addActionListener(e ->
performStep_2025());

        // Event Reset
        resetButton_2025.addActionListener(e ->
reset_2025());
    }

    private void setArrayFromInput_2025() {
        String text_2025 = inputField_2025.getText().trim();
        if (text_2025.isEmpty()) return;
        String[] parts_2025 = text_2025.split(",");
        array_2025 = new int[parts_2025.length];
        try {
            for(int k_2025 = 0; k_2025 <
parts_2025.length; k_2025++) {
                array_2025[k_2025] =
Integer.parseInt(parts_2025[k_2025].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka yang dipisahkan " + "dengan koma!",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2025 = 1;
        stepCount_2025 = 1;
        sorting_2025 = true;
        stepButton_2025.setEnabled(true);
    }

```

```

        stepArea_2025.setText("");
        panelArray_2025.removeAll();
        labelArray_2025 = new JLabel[array_2025.length];
        for (int k_2025 = 0; k_2025 < array_2025.length;
k_2025++) {
            labelArray_2025[k_2025] = new
JLabel(String.valueOf(array_2025[k_2025]));
            labelArray_2025[k_2025].setFont (new Font
("Arial", Font.BOLD, 24));

            labelArray_2025[k_2025].setBorder(BorderFactory.createLineB
order(Color.BLACK));
            labelArray_2025[k_2025].setPreferredSize (new
Dimension(50, 50));

            labelArray_2025[k_2025].setHorizontalAlignment(SwingConstan
ts.CENTER);
            panelArray_2025.add
(labelArray_2025[k_2025]);
        }
        panelArray_2025.revalidate();
        panelArray_2025.repaint();
    }
    private void performStep_2025() {
        if (i_2025 < array_2025.length &&
sorting_2025) {
            int key_2025 = array_2025[i_2025];
            j_2025 = i_2025 - 1;

            StringBuilder stepLog = new
StringBuilder();
            stepLog.append("Langkah
").append(stepCount_2025).append(": Masukkan
").append(key_2025).append("\n");

            while (j_2025 >= 0 &&
array_2025[j_2025] > key_2025) {
                array_2025[j_2025 + 1] =
array_2025[j_2025];
                j_2025--;
            }
            array_2025[j_2025 + 1] = key_2025;

            updateLabels_2025();
            stepLog.append("Hasil:
").append(arrayToString_2025(array_2025)).append("\n\n");
            stepArea_2025.append(stepLog.toString());

            i_2025++;
            stepCount_2025++;

            if (i_2025 == array_2025.length) {
                sorting_2025 = false;
                stepButton_2025.setEnabled(false);
                JOptionPane.showMessageDialog(this,"Sorting
selesai!");
            }
        }
    }

```



```

    }
}

private void updateLabels_2025() {
    for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {

        labelArray_2025[k_2025].setText(String.valueOf(array_2025[k
_2025]));
    }
}

private void reset_2025() {
    inputField_2025.setText("");
    panelArray_2025.removeAll();
    panelArray_2025.revalidate();
    panelArray_2025.repaint();
    stepArea_2025.setText("");
    stepButton_2025.setEnabled(false);
    sorting_2025 = false;
    i_2025 = 1;
    stepCount_2025 = 1;
}

private String arrayToString_2025(int[] arr) {
    StringBuilder sb_2025 = new StringBuilder();
    for (int k_2025 = 0; k_2025 < arr.length;
k_2025++) {
        sb_2025.append(arr[k_2025]);
        if (k_2025 < arr.length - 1)
sb_2025.append(", ");
    }
    return sb_2025.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511532025 gui = new
InsertionSortGUI_2511532025();
        gui.setVisible(true);
    });
}
}

```

Kode Program 2.39 Program *InsertionSortGUI*

Kode program di atas digunakan untuk menampilkan proses algoritma *Insertion Sort* secara bertahap menggunakan Java *GUI* (*Swing*). Program memungkinkan pengguna memasukkan data angka melalui *form input*, kemudian menampilkan proses pengurutan langkah demi langkah secara visual. Setiap langkah pengurutan ditampilkan pada area visual dan log proses sehingga pengguna dapat memahami alur kerja algoritma *Insertion Sort* dengan lebih jelas.

Class InsertionSortGUI_2511532025 merupakan turunan dari *class JFrame* yang digunakan untuk membuat tampilan *GUI*. Program terdiri dari beberapa komponen seperti *JTextField* untuk input data, *JButton* untuk menjalankan proses sorting, *JPanel* untuk visualisasi *array*, dan *JTextArea* untuk menampilkan langkah-langkah proses *sorting*.

Method setArrayFromInput_2025() digunakan untuk mengambil input angka dari pengguna, memisahkan data berdasarkan tanda koma, kemudian menyimpannya ke dalam array integer. Setelah data berhasil dimasukkan, program akan menampilkan setiap elemen *array* ke dalam bentuk label pada panel visualisasi.

Method performStep_2025() digunakan untuk menjalankan proses *Insertion Sort* satu langkah setiap kali tombol “Langkah Selanjutnya” ditekan. Program akan mengambil satu elemen sebagai *key_2025*, kemudian membandingkannya dengan elemen sebelumnya. Jika elemen sebelumnya lebih besar, maka elemen tersebut akan digeser ke kanan hingga ditemukan posisi yang tepat untuk *key_2025*. Setelah proses selesai, hasil sementara *array* akan ditampilkan pada area log langkah-langkah *sorting*.

Method updateLabels_2025() digunakan untuk memperbarui tampilan visual *array* pada *GUI* setelah proses sorting dilakukan. Sedangkan *method reset_2025()* digunakan untuk menghapus seluruh data *input*, visualisasi *array*, dan log langkah-langkah agar program dapat digunakan kembali dari awal.

Pada *method main()*, program dijalankan menggunakan *SwingUtilities.invokeLater()* agar tampilan *GUI* dapat berjalan dengan baik pada *event dispatch thread Swing*.

2.4.1 Langkah Kerja Program *InsertionSortGUI*

1. *Method Constructor*

```
public InsertionSortGUI_2511532025() {
```

```

        setTitle("Insertion Sort Langkah per
Langkah");
        setSize(750, 400);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
        inputField_2025 = new JTextField(30);
        setButton_2025 = new JButton("Set Array");
        inputPanel_2025.add(new JLabel("Masukkan
angka (pisahkan dengan koma):"));
        inputPanel_2025.add(inputField_2025);
        inputPanel_2025.add(setButton_2025);

        // Panel array visual
        panelArray_2025 = new JPanel();
        panelArray_2025.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2025 = new JPanel();
        stepButton_2025 = new JButton("Langkah
Selanjutnya");
        resetButton_2025 = new JButton("Reset");
        stepButton_2025.setEnabled(false);
        controlPanel_2025.add(stepButton_2025);
        controlPanel_2025.add(resetButton_2025);

        // Area teks untuk log langkah-langkah
        stepArea_2025 = new JTextArea(8, 60);
        stepArea_2025.setEditable(false);
        stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
        JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025);

        // Tambahkan panel ke frame
        add(inputPanel_2025, BorderLayout.NORTH);
        add(panelArray_2025, BorderLayout.CENTER);
        add(controlPanel_2025, BorderLayout.SOUTH);
        add(scrollPane_2025, BorderLayout.EAST);

        // Event Set Array
        setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

        // Event Langkah Selanjutnya
        stepButton_2025.addActionListener(e ->
performStep_2025());

        // Event Reset
        resetButton_2025.addActionListener(e ->
reset_2025());

```

```
}
```

Kode Program 2.40 *Constructor GUI*

Method constructor digunakan untuk membuat tampilan *GUI* program. Pada method ini dibuat panel input, panel visualisasi array, tombol kontrol, dan area log langkah *sorting*.

2. *Method setArrayFromInput_2025()*

```
private void setArrayFromInput_2025() {
    String text_2025 =
inputField_2025.getText().trim();
    if (text_2025.isEmpty()) return;
    String[] parts_2025 = text_2025.split(",");
    array_2025 = new int[parts_2025.length];
    try {
        for(int k_2025 = 0; k_2025 <
parts_2025.length; k_2025++) {
            array_2025[k_2025] =
Integer.parseInt(parts_2025[k_2025].trim());
        }
        catch (NumberFormatException e) {

            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan " + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2025 = 1;
        stepCount_2025 = 1;
        sorting_2025 = true;
        stepButton_2025.setEnabled(true);
        stepArea_2025.setText("");
        panelArray_2025.removeAll();
        labelArray_2025 = new
JLabel[array_2025.length];
        for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {
            labelArray_2025[k_2025] = new
JLabel(String.valueOf(array_2025[k_2025]));
            labelArray_2025[k_2025].setFont (new
Font ("Arial", Font.BOLD, 24));

            labelArray_2025[k_2025].setBorder(BorderFactory.crea
teLineBorder(Color.BLACK));

            labelArray_2025[k_2025].setPreferredSize (new
Dimension(50, 50));

            labelArray_2025[k_2025].setHorizontalAlignment(Swing
Constants.CENTER);
```

```

        panelArray_2025.add
(labelArray_2025[k_2025]);
    }
    panelArray_2025.revalidate();
    panelArray_2025.repaint();
}

```

Kode Program 2.41 *Method Input Array*

Method ini digunakan untuk mengambil input angka dari pengguna kemudian memisahkan data menggunakan tanda koma. Setelah itu data diubah menjadi *array integer* dan ditampilkan pada panel visualisasi.

3. *Method performStep_2025()*

```

private void performStep_2025() {
    if (i_2025 < array_2025.length &&
    sorting_2025) {
        int key_2025 =
array_2025[i_2025];
        j_2025 = i_2025 - 1;

        StringBuilder stepLog = new
StringBuilder();
        stepLog.append("Langkah
").append(stepCount_2025).append(": Memasukkan
").append(key_2025).append("\n");

        while (j_2025 >= 0 &&
array_2025[j_2025] > key_2025) {
            array_2025[j_2025 + 1] =
array_2025[j_2025];
            j_2025--;
        }
        array_2025[j_2025 + 1] = key_2025;

        updateLabels_2025();
        stepLog.append("Hasil:
").append(arrayToString_2025(array_2025)).append("\n\n
");
        stepArea_2025.append(stepLog.toString());

        i_2025++;
        stepCount_2025++;

        if (i_2025 == array_2025.length) {
            sorting_2025 = false;
            stepButton_2025.setEnabled(false);
        }
    }
}

```

```

JOptionPane.showMessageDialog(this, "Sorting
selesai!");
    }
}
}

```

Kode Program 2.42 *Method Proses Sorting*

Method ini digunakan untuk menjalankan proses *Insertion Sort* langkah demi langkah. Program akan mengambil satu elemen sebagai *key*, membandingkannya dengan elemen sebelumnya, lalu menggeser elemen yang lebih besar ke kanan hingga ditemukan posisi yang tepat.

4. *Method updateLabels_2025()*

```

private void updateLabels_2025() {
    for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {

        labelArray_2025[k_2025].setText(String.valueOf(array
_2025[k_2025]));
    }
}

```

Kode Program 2.43 *Method Update Label*

Method ini digunakan untuk memperbarui tampilan *array* pada *GUI* setelah proses *sorting* dilakukan.

5. *Method reset_2025()*

```

private void reset_2025() {
    inputField_2025.setText("");
    panelArray_2025.removeAll();
    panelArray_2025.revalidate();
    panelArray_2025.repaint();
    stepArea_2025.setText("");
    stepButton_2025.setEnabled(false);
    sorting_2025 = false;
    i_2025 = 1;
    stepCount_2025 = 1;
}

```

Kode Program 2.44 *Method Reset*

Method ini digunakan untuk menghapus input, visualisasi *array*, dan log langkah sorting sehingga program kembali ke kondisi awal.

6. *Method arrayToString_2025()*

```
private String arrayToString_2025(int[] arr)
{
    StringBuilder sb_2025 = new
    StringBuilder();
    for (int k_2025 = 0; k_2025 <
    arr.length; k_2025++) {
        sb_2025.append(arr[k_2025]);
        if (k_2025 < arr.length - 1)
    sb_2025.append(", ");
    }
    return sb_2025.toString();
}
```

Kode Program 2.45 *Method Konversi Array*

Method ini digunakan untuk mengubah isi *array* menjadi bentuk *String* agar dapat ditampilkan pada area *log sorting*.

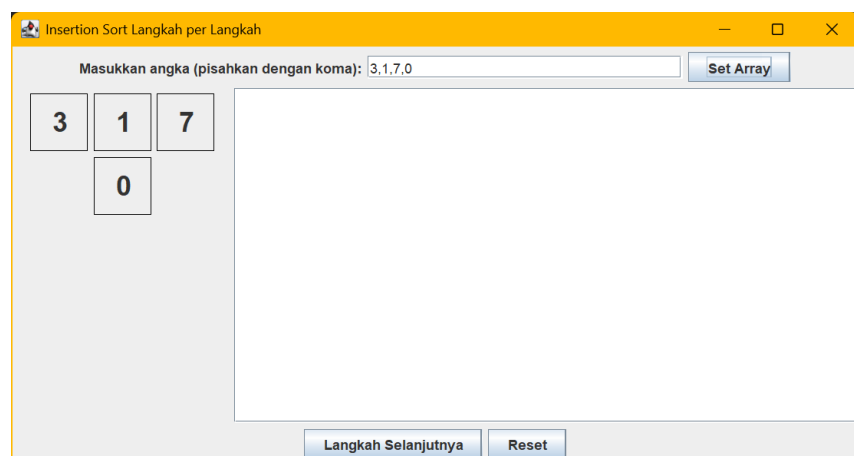
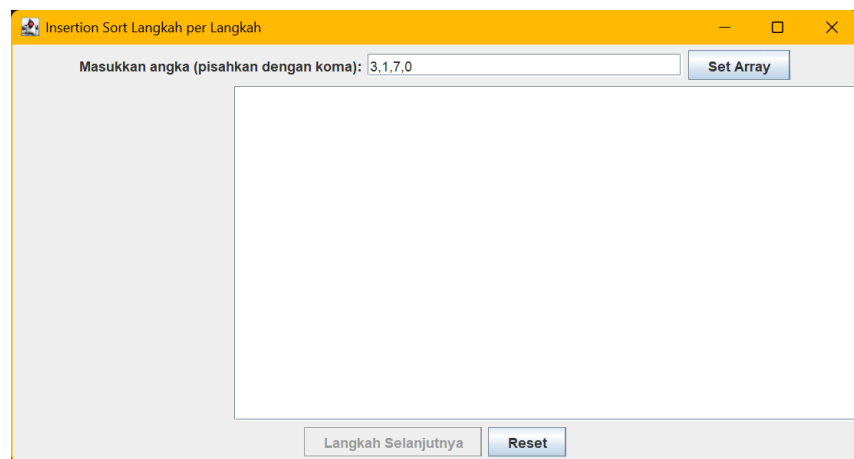
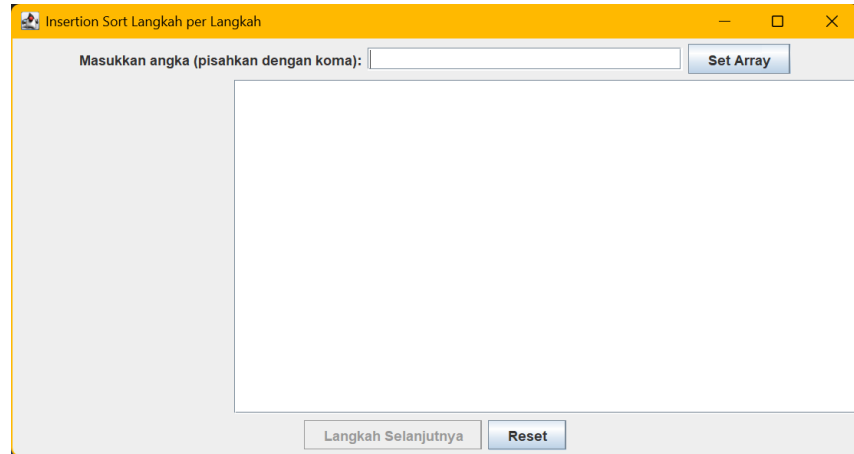
7. *Method main()*

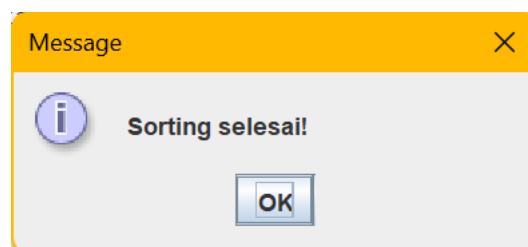
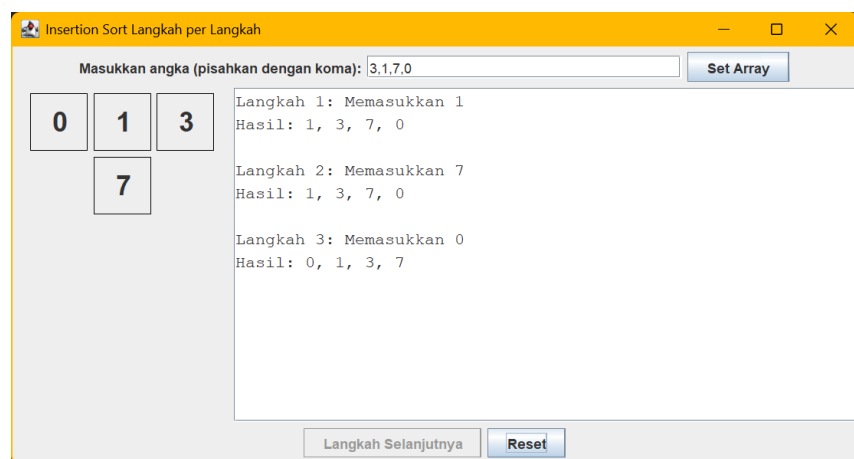
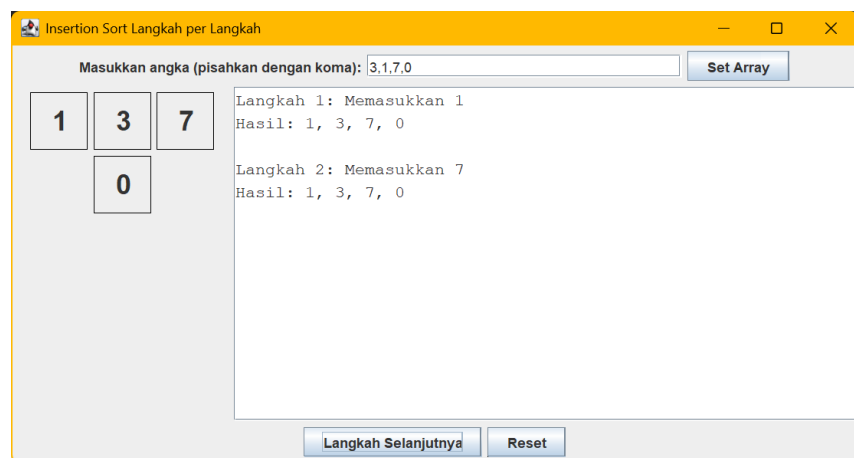
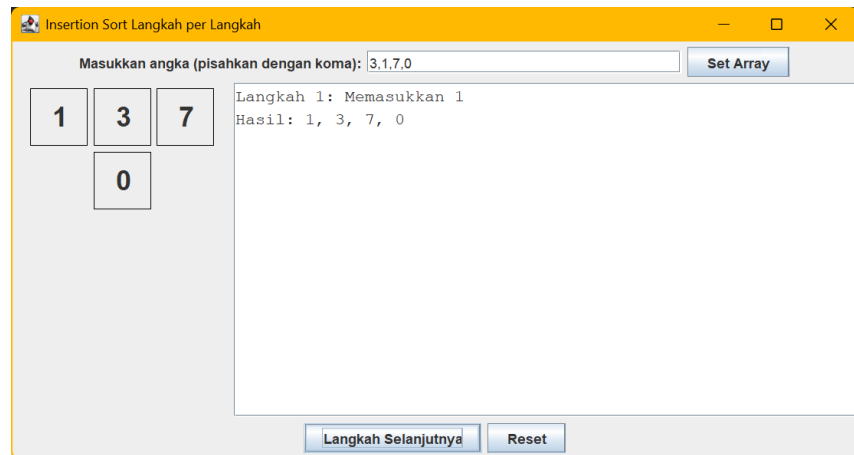
```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511532025 gui
    = new InsertionSortGUI_2511532025();
        gui.setVisible(true);
    });
}
```

Kode Program 2.46 *Method Main*

Method utama digunakan untuk menjalankan program *GUI Insertion Sort* menggunakan *SwingUtilities.invokeLater()*.

2.4.2 Hasil Program





Gambar 2.4 Hasil Program *HapusSLL*

Berdasarkan hasil eksekusi program, visualisasi algoritma *Insertion Sort* berhasil berjalan dengan baik menggunakan Java *GUI* (*Swing*). Pengguna dapat memasukkan data angka secara manual melalui form input, kemudian program akan menampilkan proses pengurutan secara bertahap setiap kali tombol “Langkah Selanjutnya” ditekan.

Program menampilkan perubahan posisi data pada setiap langkah *sorting* sehingga proses pengurutan dapat dipahami dengan lebih jelas. Selain itu, *array* juga divisualisasikan dalam bentuk label sehingga pengguna dapat melihat perubahan data secara langsung pada *GUI*.

Setelah seluruh proses selesai, data berhasil diurutkan dari nilai terkecil ke terbesar sesuai dengan konsep algoritma *Insertion Sort*. Program juga menyediakan tombol *reset* untuk menghapus data dan mengulangi proses *sorting* dari awal.

BAB III

KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan metode yang digunakan untuk mengurutkan data agar tersusun secara teratur. Pada praktikum ini digunakan tiga algoritma *sorting*, yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* yang diimplementasikan menggunakan bahasa Java.

Melalui program *IntertionSort_2511532025*, dapat dipahami bahwa algoritma Insertion Sort bekerja dengan cara mengambil satu elemen kemudian menyisipkannya pada posisi yang tepat di bagian *array* yang sudah teratur. Algoritma ini melakukan proses perbandingan dan pergeseran elemen hingga seluruh data tersusun secara menaik.

Pada program *SelectionSort_2511532025*, proses pengurutan dilakukan dengan mencari nilai terkecil pada *array* kemudian menukarnya dengan elemen pada posisi saat ini. Proses tersebut dilakukan secara berulang hingga seluruh data berhasil diurutkan.

Selanjutnya, program *BubbleSort_2511532025* menunjukkan bahwa algoritma *Bubble Sort* bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila posisi data tidak sesuai urutan. Proses ini dilakukan berulang hingga data tersusun secara teratur dari nilai terkecil ke terbesar.

Selain program berbasis *console*, praktikum ini juga mengimplementasikan Java *GUI* (*Swing*) melalui program *InsertionSortGUI_2511532025*. Program *GUI* tersebut mampu menampilkan proses sorting secara visual dan interaktif sehingga pengguna dapat melihat langkah-langkah pengurutan data satu per satu. Visualisasi ini membantu memahami alur kerja algoritma *Insertion Sort* dengan lebih jelas.

Dengan demikian, praktikum ini membantu memahami konsep dasar algoritma *sorting*, cara kerja masing-masing algoritma, serta implementasi *sorting* menggunakan Java berbasis *console* maupun *GUI*. Praktikum ini juga meningkatkan kemampuan dalam membuat program pengurutan data secara terstruktur, interaktif, dan mudah dipahami.

DAFTAR PUSTAKA

Wahyudi, S.T., M.T., *Struktur Data: Part 10 Sorting Algorithm*. Padang: Universitas Andalas, 2025.

Wahyudi, S.T., M.T., *Struktur Data: Part 11 Sorting Algorithm*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 7: Struktur Data

Topik : Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)

Tema : Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI

1. Deskripsi Program

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif.

Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).

Perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. Kode Program

1) Kelas Mahasiswa

```
package pekan7_2511532025;

public class Mahasiswa_2511532025 {

    private String nama_2025;
    private String nim_2025;
    private String prodi_2025;

    // constructor
    public Mahasiswa_2511532025(String nama_2025, String
nim_2025, String prodi_2025) {
        this.nama_2025 = nama_2025;
        this.nim_2025 = nim_2025;
        this.prodi_2025 = prodi_2025;
    }
    // getter
    public String getNama_2025() {
        return nama_2025;
    }
    public String getNim_2025() {
        return nim_2025;
    }
}
```

```

    public String getProdi_2025() {
        return prodi_2025;
    }
    // setter
    public void setNama_2025(String nama_2025) {
        this.nama_2025 = nama_2025;
    }
    public void setNim_2025(String nim_2025) {
        this.nim_2025 = nim_2025;
    }
    public void setProdi_2025(String prodi_2025) {
        this.prodi_2025 = prodi_2025;
    }
    @Override
    public String toString() {
        return nama_2025 + " - " + nim_2025 + " - " +
prodi_2025;
    }
}

```

2) Kelas GUI Mahasiswa

```

package pekan7_2511532025;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.table.DefaultTableModel;

public class MahasiswaGUI_2511532025 extends JFrame {

    private static final long serialVersionUID = 1L;

    // arraylist mahasiswa
    private ArrayList<Mahasiswa_2511532025> dataMhs_2025;

    // komponen GUI
    private JTextField txtNama_2025;
    private JTextField txtNim_2025;
    private JTextField txtProdi_2025;

    private JButton btnTambah_2025;
    private JButton btnSorting_2025;

```

```

private JButton btnReset_2025;

private JComboBox<String> comboSorting_2025;

private JTable table_2025;
private DefaultTableModel model_2025;

private JTextArea stepArea_2025;

// constructor
public MahasiswaGUI_2511532025() {

    setTitle("Sorting Nama Mahasiswa GUI");
    setSize(1000, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    dataMhs_2025 = new ArrayList<>();

    // panel input
    JPanel inputPanel_2025 = new JPanel(new
FlowLayout());

    txtNama_2025 = new JTextField(10);
    txtNim_2025 = new JTextField(10);
    txtProdi_2025 = new JTextField(10);

    btnTambah_2025 = new JButton("Tambah Data");

    inputPanel_2025.add(new JLabel("Nama"));
    inputPanel_2025.add(txtNama_2025);

    inputPanel_2025.add(new JLabel("NIM"));
    inputPanel_2025.add(txtNim_2025);

    inputPanel_2025.add(new JLabel("Prodi"));
    inputPanel_2025.add(txtProdi_2025);

    inputPanel_2025.add(btnTambah_2025);

    // tabel mahasiswa
    model_2025 = new DefaultTableModel();

    model_2025.addColumn("Nama Mahasiswa");
    model_2025.addColumn("NIM");
    model_2025.addColumn("Program Studi");

    table_2025 = new JTable(model_2025);

    JScrollPane scrollTable_2025 = new
JScrollPane(table_2025);

    // panel kontrol
    JPanel controlPanel_2025 = new JPanel();

    comboSorting_2025 = new JComboBox<>();

```



```

        comboSorting_2025.addItem("Insertion Sort");
        comboSorting_2025.addItem("Selection Sort");
        comboSorting_2025.addItem("Bubble Sort");

        btnSorting_2025 = new JButton("Mulai Sorting");
        btnReset_2025 = new JButton("Reset");

        controlPanel_2025.add(new JLabel("Pilih Sorting"));
        controlPanel_2025.add(comboSorting_2025);
        controlPanel_2025.add(btnSorting_2025);
        controlPanel_2025.add(btnReset_2025);

        // textarea langkah sorting
        stepArea_2025 = new JTextArea(15, 40);
        stepArea_2025.setEditable(false);
        stepArea_2025.setFont(
            new Font("Monospaced", Font.PLAIN, 14));

        JScrollPane scrollStep_2025 = new
        JScrollPane(stepArea_2025);

        // panel kanan
        JPanel panelKanan_2025 = new JPanel();
        panelKanan_2025.setLayout(new BorderLayout());

        JLabel lblLangkah_2025 = new JLabel("Visualisasi
        Sorting");

        lblLangkah_2025.setHorizontalAlignment(SwingConstants.CENTE
        R);

        lblLangkah_2025.setFont(new Font("Arial",
        Font.BOLD, 16));

        panelKanan_2025.add(lblLangkah_2025,
        BorderLayout.NORTH);

        panelKanan_2025.add(scrollStep_2025, BorderLayout.CENTER);

        // tambahkan ke frame
        add(inputPanel_2025, BorderLayout.NORTH);
        add(scrollTable_2025, BorderLayout.CENTER);
        add(controlPanel_2025, BorderLayout.SOUTH);
        add(panelKanan_2025, BorderLayout.EAST);

        // event tambah data
        btnTambah_2025.addActionListener(e ->
        tambahData_2025());

        // event sorting
        btnSorting_2025.addActionListener(e ->
        mulaiSorting_2025());

        // event reset

```

```

        btnReset_2025.addActionListener(e ->
resetData_2025());
    }

    // method tambah data
    private void tambahData_2025() {
        String nama_2025 = txtNama_2025.getText().trim();
        String nim_2025 = txtNim_2025.getText().trim();
        String prodi_2025 = txtProdi_2025.getText().trim();

        if (nama_2025.isEmpty() || nim_2025.isEmpty() ||
prodi_2025.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Data tidak
boleh kosong!");
            return;
        }

        Mahasiswa_2511532025 mhs_2025 = new
Mahasiswa_2511532025(nama_2025, nim_2025, prodi_2025);
        dataMhs_2025.add(mhs_2025);
        model_2025.addRow(new Object[]{nama_2025, nim_2025,
prodi_2025});
        txtNama_2025.setText("");
        txtNim_2025.setText("");
        txtProdi_2025.setText("");
    }

    // method reset
    private void resetData_2025() {
        dataMhs_2025.clear();
        model_2025.setRowCount(0);
        stepArea_2025.setText("");
    }

    // tampilkan nama
    private String tampilNama_2025() {
        String hasil_2025 = "[";
        for (int i_2025 = 0; i_2025 < dataMhs_2025.size();
i_2025++) {
            hasil_2025 +=
dataMhs_2025.get(i_2025).getNama_2025();
            if (i_2025 != dataMhs_2025.size() - 1) {
                hasil_2025 += ", ";
            }
        }
        hasil_2025 += "];";
        return hasil_2025;
    }

    // method sorting
    private void mulaiSorting_2025() {
        if (dataMhs_2025.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Data
mahasiswa kosong!");
            return;
        }
        stepArea_2025.setText("");
        String pilihan_2025 =
comboSorting_2025.getSelectedItem().toString();
        if (pilihan_2025.equals("Insertion Sort")) {

```

```

        stepArea_2025.append("=== INSERTION SORT
        ===\n");
        insertionSort_2025();
    } else if (pilihan_2025.equals("Selection
Sort")) {
        stepArea_2025.append( "=== SELECTION
SORT ===\n");
        selectionSort_2025();
    } else {
        stepArea_2025.append("=== BUBBLE SORT
        ===\n");
        bubbleSort_2025();
    }
    tampilTable_2025();
}
// tampil ulang table
private void tampilTable_2025() {
    model_2025.setRowCount(0);
    for (Mahasiswa_2511532025 mhs_2025 : dataMhs_2025) {
        model_2025.addRow(new
Object[]{mhs_2025.getNama_2025(),mhs_2025.getNim_2025(),mhs
_2025.getProdi_2025()});
    }
}
// insertion sort
private void insertionSort_2025() {
    for (int i_2025 = 1; i_2025 < dataMhs_2025.size();
i_2025++) {
        Mahasiswa_2511532025 key_2025 =
dataMhs_2025.get(i_2025);
        int j_2025 = i_2025 - 1;
        while (j_2025 >= 0 &&
dataMhs_2025.get(j_2025).getNama_2025().compareToIgnoreCase
(key_2025.getNama_2025()) > 0) {
            dataMhs_2025.set(j_2025 +
1,dataMhs_2025.get(j_2025));
            j_2025--;
        }
        dataMhs_2025.set(j_2025 + 1, key_2025);
        stepArea_2025.append("Langkah " + i_2025 + " :
" + tampilNama_2025() + "\n");
    }
}
// selection sort
private void selectionSort_2025() {
    for (int i_2025 = 0; i_2025 < dataMhs_2025.size() -
1;i_2025++) {
        int minIndex_2025 = i_2025;
        for (int j_2025 = i_2025 + 1; j_2025 <
dataMhs_2025.size(); j_2025++) {
            if
(dataMhs_2025.get(j_2025).getNama_2025().compareToIgnoreCas
e(dataMhs_2025.get(minIndex_2025).getNama_2025()) < 0) {
                minIndex_2025 = j_2025;
            }
        }
    }
}

```

```

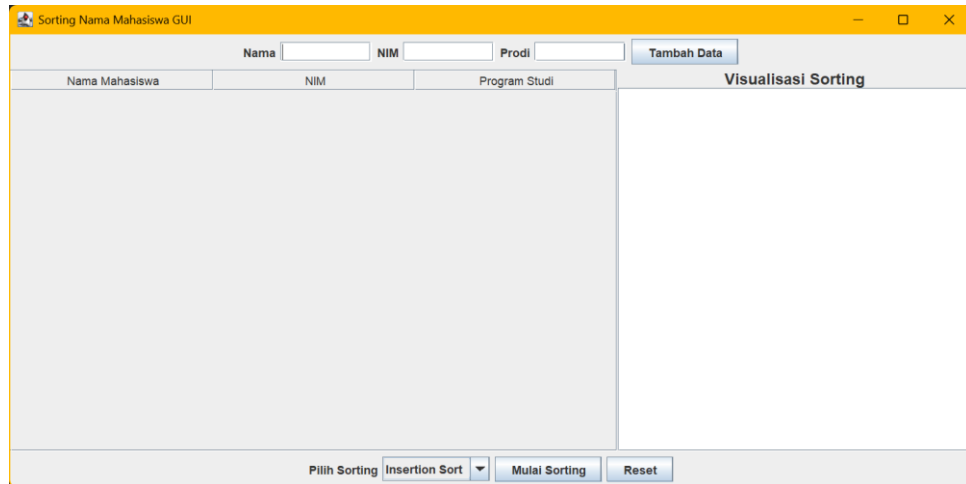
        Mahasiswa_2511532025 temp_2025 =
dataMhs_2025.get(i_2025);
        dataMhs_2025.set(i_2025,
dataMhs_2025.get(minIndex_2025));
        dataMhs_2025.set(minIndex_2025, temp_2025);
        stepArea_2025.append("Pass " + (i_2025 + 1) +
" : " + tampilNama_2025() + "\n");
    }
}
// bubble sort
private void bubbleSort_2025() {
    for (int i_2025 = 0; i_2025 < dataMhs_2025.size() -
1; i_2025++) {
        for (int j_2025 = 0; j_2025 <
dataMhs_2025.size() - i_2025 - 1; j_2025++) {
            if
(dataMhs_2025.get(j_2025).getNama_2025().compareToIgnoreCas
e(dataMhs_2025.get(j_2025 + 1).getNama_2025()) > 0) {
                Mahasiswa_2511532025 temp_2025 =
dataMhs_2025.get(j_2025);

dataMhs_2025.set(j_2025, dataMhs_2025.get(j_2025 + 1));
                dataMhs_2025.set(j_2025 + 1,
temp_2025);
            }
        }
        stepArea_2025.append("Pass " + (i_2025 + 1) +
" : " + tampilNama_2025() + "\n");
    }
}
// main
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MahasiswaGUI_2511532025 gui_2025 = new
MahasiswaGUI_2511532025 ();
        gui_2025.setVisible(true);
    });
}
}

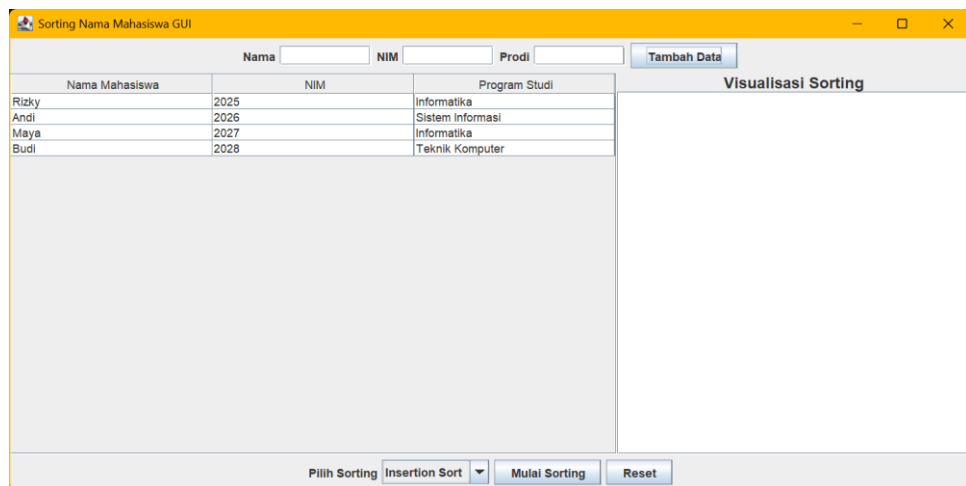
```

3. Tampilan Output Program

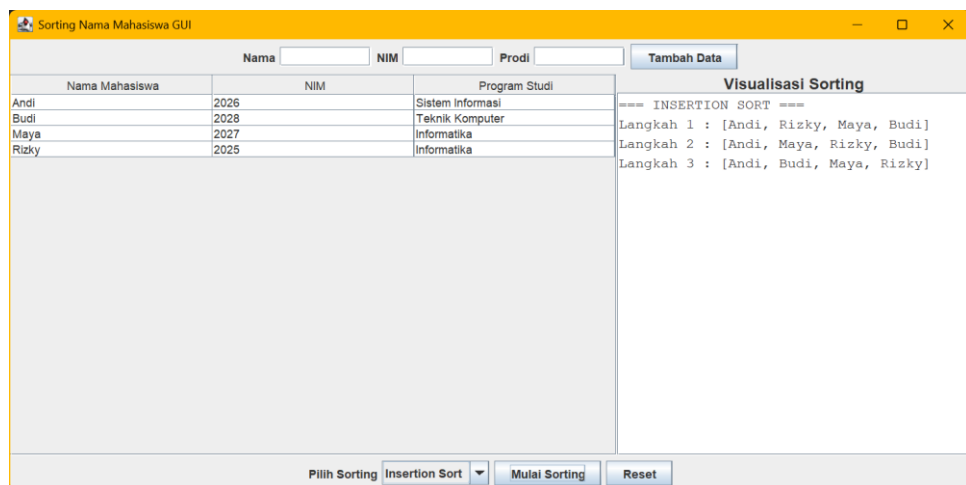
Tampilan awal GUI



Tampilan saat data sudah diinput



Tampilan Insertion Sort



Tampilan Selection Sort

The screenshot shows a Java Swing window titled "Sorting Nama Mahasiswa GUI". At the top, there are input fields for "Nama", "NIM", and "Prodi", followed by a "Tambah Data" button. Below these is a table with three columns: "Nama Mahasiswa", "NIM", and "Program Studi". The table contains the following data:

Nama Mahasiswa	NIM	Program Studi
Andi	2026	Sistem Informasi
Budi	2028	Teknik Komputer
Maya	2027	Informatika
Rizky	2025	Informatika

To the right of the table is a text area titled "Visualisasi Sorting" which displays the output of the Selection Sort algorithm:

```
=== SELECTION SORT ===  
Pass 1 : [Andi, Budi, Maya, Rizky]  
Pass 2 : [Andi, Budi, Maya, Rizky]  
Pass 3 : [Andi, Budi, Maya, Rizky]
```

At the bottom of the window, there is a "Pilih Sorting" dropdown menu set to "Selection Sort", and two buttons: "Mulai Sorting" and "Reset".

Tampilan Bubble Sort

This screenshot is identical to the one above, but the "Pilih Sorting" dropdown menu is set to "Bubble Sort". The "Visualisasi Sorting" text area now displays the output of the Bubble Sort algorithm:

```
=== BUBBLE SORT ===  
Pass 1 : [Andi, Budi, Maya, Rizky]  
Pass 2 : [Andi, Budi, Maya, Rizky]  
Pass 3 : [Andi, Budi, Maya, Rizky]
```

Tampilan ketika data direset

This screenshot shows the same GUI, but the data table is empty. The "Visualisasi Sorting" text area is also empty. The "Pilih Sorting" dropdown menu is still set to "Bubble Sort", and the "Mulai Sorting" and "Reset" buttons are visible at the bottom.